Java Coding Interview

Complete Question Bank

8 Years Experience · Service-Based & Product Companies

Covers Every Pattern to Clear Any Interview

TCS · Infosys · Wipro · Cognizant · Accenture · HCL · Capgemini · Tech Mahindra IBM · LTIMindtree · Mphasis · Hexaware · NIIT Technologies

100+ curated questions · Every major DSA pattern · Complexity analysis · Java 8+ solutions

■ Table of Contents

1. String & Array Manipulation

Q1. Reverse a String without using built-in reverse().

■ **Asked at: TCS · Infosys · Wipro · Accenture**

■ **Pattern: Two Pointer**

Use two-pointer swap from both ends, or convert to char array and iterate.

■ **Time O(n) | Space O(n) for char array**

```
public String reverse(String s) {
char[] ch = s.toCharArray();
int l = 0, r = ch.length - 1;
while (l < r) {
char tmp = ch[l]; ch[l] = ch[r]; ch[r] = tmp;
l++; r--;
}
return new String(ch);
}
```

✔ *Tip: Java Streams: new StringBuilder(s).reverse().toString() — mention but implement manually.*

Q2. Check if a String is a palindrome (ignore spaces & case).

■ **Asked at: Cognizant · HCL · Capgemini · TCS**

■ **Pattern: Two Pointer**

Strip non-alphanumeric, lowercase, then compare with reversed version.

■ **Time O(n) | Space O(n)**

```
public boolean isPalindrome(String s) {
String clean = s.replaceAll("[^a-zA-Z0-9]", "").toLowerCase();
int l = 0, r = clean.length() - 1;
while (l < r) {
if (clean.charAt(l++) != clean.charAt(r--)) return false;
}
return true;
}
```

Q3. Find the first non-repeating character in a String.

■ **Asked at: TCS · Infosys · IBM · Mphasis**

■ **Pattern: Hashing / Frequency Count**

Use a LinkedHashMap to preserve insertion order while counting frequency.

■ **Time O(n) | Space O(1) — at most 26 distinct chars**

```
public char firstUnique(String s) {
Map<Character, Integer> freq = new LinkedHashMap<>();
for (char c : s.toCharArray())
freq.merge(c, 1, Integer::sum);
for (Map.Entry<Character, Integer> e : freq.entrySet())
if (e.getValue() == 1) return e.getKey();
return '\0';
}
// "aabbcde" -> c
```

Q4. Count occurrences of each character in a String.

■ **Asked at: Wipro · Accenture · Capgemini**

■ **Pattern: Frequency Count**

int[256] for ASCII frequency, or Map for Unicode.

■ **Time O(n) | Space O(k) where k = distinct chars**

```
public Map<Character,Long> charFreq(String s) {
return s.chars()
.mapToObj(c -> (char) c)
.collect(Collectors.groupingBy(c -> c, Collectors.counting()));
}
```

Q5. Check if two Strings are anagrams.

■ **Asked at: TCS · Infosys · HCL · Tech Mahindra**

■ **Pattern: Frequency Count**

Sort both and compare, or use frequency array.

■ **Time O(n) | Space O(1)**

```
public boolean isAnagram(String a, String b) {
if (a.length() != b.length()) return false;
int[] freq = new int[26];
for (int i = 0; i < a.length(); i++) {
freq[a.charAt(i) - 'a']++;
freq[b.charAt(i) - 'a']--;
}
for (int f : freq) if (f != 0) return false;
return true;
}
```

✔ *Tip: For Unicode strings use a HashMap instead of int[26].*

Q6. Find the longest substring without repeating characters.

■ **Asked at: TCS Digital · Infosys SP · Cognizant GenC Pro · IBM**

■ **Pattern: Sliding Window**

Sliding window with a HashSet. Shrink window from left when duplicate found.

■ **Time O(n) | Space O(min(n,k))**

```
public int lengthOfLongestSubstring(String s) {
Set<Character> set = new HashSet<>();
int l = 0, max = 0;
for (int r = 0; r < s.length(); r++) {
while (set.contains(s.charAt(r)))
set.remove(s.charAt(l++));
set.add(s.charAt(r));
max = Math.max(max, r - l + 1);
}
return max;
}
// "abcabcbb" -> 3 ("abc")
```

Q7. Find all permutations of a String.

■ **Asked at: Wipro Elite · TCS Digital · Accenture**

■ **Pattern: Backtracking**

Recursion + swap: fix one char, recurse on remaining.

■ **Time O(n·n!) | Space O(n)**

```
public void permute(char[] ch, int start, List<String> res) {
if (start == ch.length) { res.add(new String(ch)); return; }
for (int i = start; i < ch.length; i++) {
swap(ch, start, i);
permute(ch, start + 1, res);
swap(ch, start, i); // backtrack
}
}
private void swap(char[] ch, int i, int j)
{ char t = ch[i]; ch[i] = ch[j]; ch[j] = t; }
```

Q8. Find the maximum sum subarray (Kadane's Algorithm).

■ **Asked at: TCS · Infosys · Wipro · Cognizant · HCL · All Service Co.**

■ **Pattern: Dynamic Programming / Greedy**

Track current sum and reset to 0 when it goes negative.

■ **Time O(n) | Space O(1)**

```
public int maxSubArray(int[] nums) {
int maxSum = nums[0], curr = nums[0];
for (int i = 1; i < nums.length; i++) {
curr = Math.max(nums[i], curr + nums[i]);
maxSum = Math.max(maxSum, curr);
}
return maxSum;
}
// [-2,1,-3,4,-1,2,1,-5,4] -> 6
```

✔ *Tip: Must-know for every service company. Also be ready to print the subarray indices.*

Q9. Rotate an array to the right by k steps.

■ **Asked at: Infosys · Accenture · Capgemini · TCS**

■ **Pattern: Array / Two Pointer**

Reverse three times: full array, first k, last n-k.

■ **Time O(n) | Space O(1)**

```
public void rotate(int[] nums, int k) {
int n = nums.length;
k %= n;
reverse(nums, 0, n - 1);
reverse(nums, 0, k - 1);
reverse(nums, k, n - 1);
}
void reverse(int[] a, int l, int r)
{ while (l<r) { int t=a[l];a[l]=a[r];a[r]=t; l++;r--; } }
// [1,2,3,4,5,6,7], k=3 -> [5,6,7,1,2,3,4]
```

Q10. Find two numbers in an array that add up to a target (Two Sum).

■ **Asked at: TCS NQT · Infosys · Wipro · Cognizant · HCL · IBM**

■ **Pattern: Hashing**

HashMap stores value -> index. One pass: check complement existence.

■ **Time O(n) | Space O(n)**

```
public int[] twoSum(int[] nums, int target) {
Map<Integer,Integer> map = new HashMap<>();
for (int i = 0; i < nums.length; i++) {
int comp = target - nums[i];
if (map.containsKey(comp))
return new int[]{map.get(comp), i};
map.put(nums[i], i);
}
return new int[]{};
}
```

✔ *Tip: Also practice 3-Sum (sorted + two pointer) for follow-ups.*

Q11. Move all zeros to the end of an array while maintaining order of non-zero elements.

■ **Asked at: TCS · Wipro · Accenture · Tech Mahindra**

■ **Pattern: Two Pointer**

Two pointer: write pointer tracks next non-zero position.

■ **Time O(n) | Space O(1)**

```
public void moveZeroes(int[] nums) {
int write = 0;
for (int num : nums)
if (num != 0) nums[write++] = num;
while (write < nums.length) nums[write++] = 0;
}
// [0,1,0,3,12] -> [1,3,12,0,0]
```

Q12. Find the missing number in an array of 1 to N.

■ **Asked at: All Service Companies — very frequent**

■ **Pattern: Math / Bit Manipulation**

Expected sum formula: N*(N+1)/2 minus actual sum. Or XOR approach.

■ **Time O(n) | Space O(1)**

```
public int missingNumber(int[] nums) {
int n = nums.length;
int expected = n * (n + 1) / 2;
int actual = Arrays.stream(nums).sum();
return expected - actual;
}
// XOR approach (handles overflow for large N):
public int missingXOR(int[] nums) {
int xor = nums.length;
for (int i = 0; i < nums.length; i++) xor ^= i ^ nums[i];
return xor;
}
```

✔ *Tip: Always mention both approaches — interviewers love the XOR trick.*

Q13. Find the duplicate number in an array of 1 to N (Floyd's Cycle Detection).

■ **Asked at: TCS Digital · Infosys SP · Cognizant GenC Pro**

■ **Pattern: Floyd's Cycle Detection**

Treat array values as 'next' pointers — the cycle entry point is the duplicate.

■ **Time O(n) | Space O(1)**

```
public int findDuplicate(int[] nums) {
int slow = nums[0], fast = nums[0];
do {
slow = nums[slow];
fast = nums[nums[fast]];
} while (slow != fast);
slow = nums[0];
while (slow != fast) {
slow = nums[slow];
fast = nums[fast];
}
return slow;
}
```

✔ *Tip: No extra space, no modifying array — that's what makes this a 'Hard'.*

Q14. Find all pairs in an array with a given difference k.

■ **Asked at: Infosys · Wipro · HCL · Mphasis**

■ **Pattern: Hashing**

Sort + two pointer OR use a HashSet for O(n) solution.

■ **Time O(n) | Space O(n)**

```
public List<int[]> pairsWithDiff(int[] arr, int k) {
Set<Integer> set = new HashSet<>();
List<int[]> result = new ArrayList<>();
for (int n : arr) set.add(n);
for (int n : arr) {
if (set.contains(n + k)) result.add(new int[]{n, n + k});
if (set.contains(n - k)) result.add(new int[]{n - k, n});
}
return result;
}
```

Q15. Find the maximum product subarray.

■ **Asked at: TCS Digital · Cognizant · IBM**

■ **Pattern: Dynamic Programming**

Track both max and min at each step (negative * negative = positive).

■ **Time O(n) | Space O(1)**

```
public int maxProduct(int[] nums) {
int max = nums[0], min = nums[0], result = nums[0];
for (int i = 1; i < nums.length; i++) {
int tmp = max;
max = Math.max(nums[i], Math.max(max * nums[i], min * nums[i]));
min = Math.min(nums[i], Math.min(tmp * nums[i], min * nums[i]));
result = Math.max(result, max);
}
return result;
}
// [2,3,-2,4] -> 6
```

---

2. Searching & Sorting

Q16. Implement Binary Search on a sorted array.

■ **Asked at: TCS NQT · Infosys · Wipro · Cognizant · All Companies**

■ **Pattern: Binary Search**

Maintain lo/hi pointers; mid = lo + (hi-lo)/2 to avoid integer overflow.

■ **Time O(log n) | Space O(1)**

```
public int binarySearch(int[] arr, int target) {
int lo = 0, hi = arr.length - 1;
while (lo <= hi) {
int mid = lo + (hi - lo) / 2;
if (arr[mid] == target) return mid;
else if (arr[mid] < target) lo = mid + 1;
else hi = mid - 1;
}
return -1;
}
```

✔ *Tip: Always use lo + (hi-lo)/2 — prevents overflow for large arrays.*

Q17. Search in a rotated sorted array.

■ **Asked at: TCS Digital · Infosys SP · Cognizant GenC Pro · IBM**

■ **Pattern: Binary Search**

Modified binary search: identify which half is sorted, then decide which side to search.

■ **Time O(log n) | Space O(1)**

```
public int search(int[] nums, int target) {
int lo = 0, hi = nums.length - 1;
while (lo <= hi) {
int mid = lo + (hi - lo) / 2;
if (nums[mid] == target) return mid;
if (nums[lo] <= nums[mid]) { // left half sorted
if (nums[lo] <= target && target < nums[mid]) hi = mid - 1;
else lo = mid + 1;
} else { // right half sorted
if (nums[mid] < target && target <= nums[hi]) lo = mid + 1;
else hi = mid - 1;
}
}
return -1;
}
```

Q18. Find the first and last position of a target in a sorted array.

■ **Asked at: TCS Digital · Wipro Elite · HCL**

■ **Pattern: Binary Search**

Binary search twice — once to find leftmost, once for rightmost position.

■ **Time O(log n) | Space O(1)**

```
public int[] searchRange(int[] nums, int t) {
return new int[]{find(nums,t,true), find(nums,t,false)};
}
int find(int[] a, int t, boolean first) {
int lo=0,hi=a.length-1,res=-1;
while(lo<=hi){
int mid=lo+(hi-lo)/2;
if(a[mid]==t){res=mid; if(first) hi=mid-1; else lo=mid+1;}
else if(a[mid]<t) lo=mid+1;
else hi=mid-1;
}
return res;
}
// [5,7,7,8,8,10], target=8 -> [3,4]
```

Q19. Implement Merge Sort.

■ **Asked at: Infosys · TCS · Accenture · Cognizant**

■ **Pattern: Divide & Conquer**

Divide array in half, sort each, merge. Stable sort O(n log n).

■ **Time O(n log n) | Space O(n)**

```
public void mergeSort(int[] arr, int l, int r) {
if (l >= r) return;
int mid = l + (r - l) / 2;
mergeSort(arr, l, mid);
mergeSort(arr, mid+1, r);
merge(arr, l, mid, r);
}
void merge(int[] a, int l, int m, int r) {
int[] tmp = Arrays.copyOfRange(a, l, r+1);
int i=0,j=m-l+1,k=l;
while(i<=m-l && j<=r-l) a[k++]=(tmp[i]<=tmp[j])?tmp[i++]:tmp[j++];
while(i<=m-l) a[k++]=tmp[i++];
while(j<=r-l) a[k++]=tmp[j++];
}
```

Q20. Implement Quick Sort.

■ **Asked at: Wipro · TCS · IBM · Mphasis**

■ **Pattern: Divide & Conquer**

Partition around a pivot; recursively sort left and right subarrays.

■ **Time O(n log n) avg, O(n²) worst | Space O(log n) stack**

```
public void quickSort(int[] arr, int lo, int hi) {
if (lo < hi) {
int p = partition(arr, lo, hi);
quickSort(arr, lo, p - 1);
quickSort(arr, p + 1, hi);
}
}
int partition(int[] a, int lo, int hi) {
int pivot = a[hi], i = lo - 1;
for (int j = lo; j < hi; j++)
if (a[j] <= pivot) { i++; int t=a[i];a[i]=a[j];a[j]=t; }
int t=a[i+1];a[i+1]=a[hi];a[hi]=t;
return i + 1;
}
```

✔ *Tip: Use random pivot selection in production to avoid O(n²) on sorted input.*

Q21. Find the Kth largest element in an unsorted array.

■ **Asked at: TCS Digital · Infosys SP · Cognizant GenC Pro · IBM**

■ **Pattern: Heap / QuickSelect**

Use a min-heap of size k. Or QuickSelect for O(n) average.

■ **Time O(n log k) | Space O(k)**

```
// Min-Heap O(n log k)
public int findKthLargest(int[] nums, int k) {
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
for (int n : nums) {
minHeap.offer(n);
if (minHeap.size() > k) minHeap.poll();
}
return minHeap.peek();
}
// [3,2,1,5,6,4], k=2 -> 5
```

✔ *Tip: QuickSelect gives O(n) avg but O(n²) worst — mention trade-offs.*

Q22. Sort an array of 0s, 1s, and 2s (Dutch National Flag).

■ **Asked at: TCS · Infosys · Wipro · Accenture · HCL**

■ **Pattern: Three Pointer**

Three pointers: lo, mid, hi. Single pass partition.

■ **Time O(n) | Space O(1)**

```
public void sortColors(int[] nums) {
int lo = 0, mid = 0, hi = nums.length - 1;
while (mid <= hi) {
if (nums[mid] == 0) swap(nums, lo++, mid++);
else if (nums[mid] == 1) mid++;
else swap(nums, mid, hi--);
}
}
// [2,0,2,1,1,0] -> [0,0,1,1,2,2]
```

✔ *Tip: Also known as 3-way partitioning — key concept for QuickSort optimization.*

Q23. Find the peak element in an array (element greater than its neighbours).

■ **Asked at: TCS Digital · Cognizant**

■ **Pattern: Binary Search**

Binary search: if mid > mid+1 then peak is in left half, else right half.

■ **Time O(log n) | Space O(1)**

```
public int findPeakElement(int[] nums) {
int lo = 0, hi = nums.length - 1;
while (lo < hi) {
int mid = lo + (hi - lo) / 2;
if (nums[mid] > nums[mid + 1]) hi = mid;
else lo = mid + 1;
}
return lo;
}
```

Q24. Merge overlapping intervals.

■ **Asked at: TCS Digital · Infosys SP · IBM · Wipro Elite**

■ **Pattern: Sorting + Greedy**

Sort by start time. Merge current interval with last result if overlap.

■ **Time O(n log n) | Space O(n)**

```
public int[][] merge(int[][] intervals) {
Arrays.sort(intervals, (a,b) -> a[0] - b[0]);
List<int[]> res = new ArrayList<>();
for (int[] iv : intervals) {
if (res.isEmpty() || res.get(res.size()-1)[1] < iv[0])
res.add(iv);
else
res.get(res.size()-1)[1] = Math.max(res.get(res.size()-1)[1], iv[1]);
}
return res.toArray(new int[0][]);
}
// [[1,3],[2,6],[8,10],[15,18]] -> [[1,6],[8,10],[15,18]]
```

---

3. Linked List

Q25. Reverse a Singly Linked List.

■ **Asked at: TCS · Infosys · Wipro · Cognizant · Accenture · All Service Co.**

■ **Pattern: Linked List / Pointer Manipulation**

Iterative: maintain prev, curr, next pointers. Classic must-know.

■ **Time O(n) | Space O(1) iterative, O(n) recursive**

```
public ListNode reverse(ListNode head) {
ListNode prev = null, curr = head;
while (curr != null) {
ListNode next = curr.next;
curr.next = prev;
prev = curr;
curr = next;
}
return prev;
}
```

✔ *Tip: Also implement recursively — different interviewer, different preference.*

Q26. Detect a cycle in a Linked List (Floyd's Algorithm).

■ **Asked at: TCS NQT · Infosys · Wipro · Cognizant · HCL**

■ **Pattern: Floyd's Cycle Detection**

Slow pointer moves 1 step, fast pointer moves 2 steps. They meet if cycle exists.

■ **Time O(n) | Space O(1)**

```
public boolean hasCycle(ListNode head) {
ListNode slow = head, fast = head;
while (fast != null && fast.next != null) {
slow = slow.next;
fast = fast.next.next;
if (slow == fast) return true;
}
return false;
}
```

✔ *Tip: Follow-up: find the start of the cycle — reset slow to head, advance both 1 step.*

Q27. Find the middle node of a Linked List.

■ **Asked at: TCS · Infosys · Accenture · Tech Mahindra**

■ **Pattern: Slow/Fast Pointer**

Slow/fast pointer — when fast reaches end, slow is at middle.

■ **Time O(n) | Space O(1)**

```
public ListNode middleNode(ListNode head) {
ListNode slow = head, fast = head;
while (fast != null && fast.next != null) {
slow = slow.next;
fast = fast.next.next;
}
return slow;
}
// 1->2->3->4->5 -> returns node 3
```

Q28. Remove Nth node from the end of a Linked List.

■ **Asked at: TCS Digital · Cognizant GenC Pro · IBM**

■ **Pattern: Two Pointer**

Two pointers gap of N: advance fast N steps, then move both until fast reaches end.

■ **Time O(n) | Space O(1)**

```
public ListNode removeNthFromEnd(ListNode head, int n) {
ListNode dummy = new ListNode(0);
dummy.next = head;
ListNode slow = dummy, fast = dummy;
for (int i = 0; i <= n; i++) fast = fast.next;
while (fast != null) { slow = slow.next; fast = fast.next; }
slow.next = slow.next.next;
return dummy.next;
}
```

Q29. Merge two sorted Linked Lists.

■ **Asked at: TCS · Infosys · Wipro · HCL · Capgemini**

■ **Pattern: Two Pointer / Merge**

Compare heads, attach smaller, recurse or iterate.

■ **Time O(m+n) | Space O(1)**

```
public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
ListNode dummy = new ListNode(0), cur = dummy;
while (l1 != null && l2 != null) {
if (l1.val <= l2.val) { cur.next = l1; l1 = l1.next; }
else { cur.next = l2; l2 = l2.next; }
cur = cur.next;
}
cur.next = (l1 != null) ? l1 : l2;
return dummy.next;
}
```

Q30. Check if a Linked List is a palindrome.

■ **Asked at: TCS Digital · Wipro Elite · Infosys SP**

■ **Pattern: Slow/Fast Pointer + Reverse**

Find middle -> reverse second half -> compare both halves -> restore.

■ **Time O(n) | Space O(1)**

```
public boolean isPalindrome(ListNode head) {
ListNode slow = head, fast = head;
while (fast != null && fast.next != null)
{ slow = slow.next; fast = fast.next.next; }
ListNode rev = reverse(slow);
ListNode p = head, q = rev;
while (q != null) {
if (p.val != q.val) return false;
p = p.next; q = q.next;
}
return true;
}
```

Q31. Add two numbers represented as Linked Lists (digits in reverse).

■ **Asked at: TCS Digital · Cognizant · IBM · Mphasis**

■ **Pattern: Simulation**

Simulate addition digit-by-digit maintaining carry.

■ **Time O(max(m,n)) | Space O(max(m,n))**

```
public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
ListNode dummy = new ListNode(0), cur = dummy;
int carry = 0;
while (l1!=null || l2!=null || carry!=0) {
int sum = carry;
if (l1!=null) { sum+=l1.val; l1=l1.next; }
if (l2!=null) { sum+=l2.val; l2=l2.next; }
carry = sum / 10;
cur.next = new ListNode(sum % 10);
cur = cur.next;
}
return dummy.next;
}
// 342+465=807 represented as linked lists
```

Q32. Implement an LRU Cache using LinkedHashMap.

■ **Asked at: TCS Digital · Infosys SP · HCL · IBM**

■ **Pattern: Design / Linked List**

Extend LinkedHashMap with removeEldestEntry — one-liner eviction policy.

■ **Time O(1) get/put | Space O(cap)**

```
class LRUCache {
private final int cap;
private final LinkedHashMap<Integer,Integer> cache;
LRUCache(int cap) {
this.cap = cap;
cache = new LinkedHashMap<>(cap, 0.75f, true) { // accessOrder=true
protected boolean removeEldestEntry(Map.Entry<Integer,Integer> e)
{ return size() > cap; }
};
}
public int get(int k) { return cache.getOrDefault(k,-1); }
public void put(int k, int v) { cache.put(k,v); }
}
```

✔ *Tip: For manual implementation (no LinkedHashMap), use HashMap + Doubly Linked List.*

---

4. Stack & Queue

Q33. Valid Parentheses — check if brackets are balanced.

■ **Asked at: TCS NQT · Infosys · Wipro · Accenture · All Service Co.**

■ **Pattern: Stack**

Push open brackets; on close bracket check/pop matching open.

■ **Time O(n) | Space O(n)**

```
public boolean isValid(String s) {
Deque<Character> stack = new ArrayDeque<>();
for (char c : s.toCharArray()) {
if ("({[".indexOf(c) >= 0) stack.push(c);
else {
if (stack.isEmpty()) return false;
char top = stack.pop();
if ((c==')' && top!='(') || (c=='}' && top!='{') ||
(c==']' && top!='[')) return false;
}
}
return stack.isEmpty();
}
```

Q34. Implement a Stack that supports push, pop, top, and getMin in O(1).

■ **Asked at: TCS Digital · Cognizant GenC Pro · IBM · Infosys SP**

■ **Pattern: Stack Design**

Use an auxiliary min-stack. Push to min-stack when new val <= current min.

■ **Time O(1) all ops | Space O(n)**

```
class MinStack {
Deque<Integer> stack = new ArrayDeque<>();
Deque<Integer> minStack = new ArrayDeque<>();
public void push(int val) {
stack.push(val);
if (minStack.isEmpty() || val <= minStack.peek())
minStack.push(val);
}
public void pop() {
if (stack.pop().equals(minStack.peek())) minStack.pop();
}
public int top() { return stack.peek(); }
public int getMin() { return minStack.peek(); }
}
```

Q35. Evaluate a Reverse Polish Notation (postfix) expression.

■ **Asked at: TCS Digital · Wipro Elite · Infosys**

■ **Pattern: Stack**

Push operands; on operator pop two, compute, push result.

■ **Time O(n) | Space O(n)**

```
public int evalRPN(String[] tokens) {
Deque<Integer> stack = new ArrayDeque<>();
for (String t : tokens) {
if ("+-*/".contains(t)) {
int b = stack.pop(), a = stack.pop();
switch(t) {
case "+": stack.push(a+b); break;
case "-": stack.push(a-b); break;
case "*": stack.push(a*b); break;
case "/": stack.push(a/b); break;
}
} else stack.push(Integer.parseInt(t));
}
return stack.pop();
}
// ["2","1","+","3","*"] -> 9 ((2+1)*3)
```

Q36. Implement a Queue using two Stacks.

■ **Asked at: TCS · Infosys · Accenture · Capgemini**

■ **Pattern: Stack / Queue Design**

Enqueue to stack1. Dequeue: if stack2 empty, pour all of stack1 into stack2.

■ **Amortized O(1) per op | Space O(n)**

```
class MyQueue {
Deque<Integer> in = new ArrayDeque<>(), out = new ArrayDeque<>();
public void push(int x) { in.push(x); }
public int pop() { move(); return out.pop(); }
public int peek() { move(); return out.peek(); }
public boolean empty() { return in.isEmpty() && out.isEmpty(); }
private void move() {
if (out.isEmpty())
while (!in.isEmpty()) out.push(in.pop());
}
}
```

Q37. Find the next greater element for each element in an array.

■ **Asked at: TCS Digital · Cognizant · HCL · Wipro Elite**

■ **Pattern: Monotonic Stack**

Monotonic decreasing stack. For each element, pop all smaller elements and assign answer.

■ **Time O(n) | Space O(n)**

```
public int[] nextGreaterElement(int[] nums) {
int n = nums.length;
int[] res = new int[n];
Arrays.fill(res, -1);
Deque<Integer> stack = new ArrayDeque<>(); // stores indices
for (int i = 0; i < n; i++) {
while (!stack.isEmpty() && nums[stack.peek()] < nums[i])
res[stack.pop()] = nums[i];
stack.push(i);
}
return res;
}
// [2,1,2,4,3] -> [4,2,4,-1,-1]
```

Q38. Largest rectangle in a histogram.

■ **Asked at: TCS Digital · Infosys SP · Cognizant GenC Pro**

■ **Pattern: Monotonic Stack**

Monotonic stack: maintain increasing bar indices. On each pop, calculate area.

■ **Time O(n) | Space O(n)**

```
public int largestRectangleArea(int[] heights) {
Deque<Integer> stack = new ArrayDeque<>();
int max = 0;
int[] h = Arrays.copyOf(heights, heights.length + 1); // sentinel 0
for (int i = 0; i < h.length; i++) {
while (!stack.isEmpty() && h[stack.peek()] > h[i]) {
int height = h[stack.pop()];
int width = stack.isEmpty() ? i : i - stack.peek() - 1;
max = Math.max(max, height * width);
}
stack.push(i);
}
return max;
}
// [2,1,5,6,2,3] -> 10
```

✔ *Tip: The sentinel 0 at the end forces all remaining stack elements to be processed.*

Q39. Sliding window maximum using a Deque.

■ **Asked at: TCS Digital · Infosys SP**

■ **Pattern: Monotonic Deque / Sliding Window**

Maintain a monotonic decreasing deque of indices. Front is always the max.

■ **Time O(n) | Space O(k)**

```
public int[] maxSlidingWindow(int[] nums, int k) {
int n = nums.length;
int[] res = new int[n - k + 1];
Deque<Integer> dq = new ArrayDeque<>();
for (int i = 0; i < n; i++) {
if (!dq.isEmpty() && dq.peekFirst() < i - k + 1) dq.pollFirst();
while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i]) dq.pollLast();
dq.offerLast(i);
if (i >= k - 1) res[i - k + 1] = nums[dq.peekFirst()];
}
return res;
}
// [1,3,-1,-3,5,3,6,7], k=3 -> [3,3,5,5,6,7]
```

Q40. Design a Circular Queue.

■ **Asked at: Accenture · Capgemini · Wipro · HCL**

■ **Pattern: Queue Design**

Array-based ring buffer with head, tail pointers and size counter.

■ **Time O(1) all ops | Space O(k)**

```
class MyCircularQueue {
int[] data; int head, tail, size, cap;
MyCircularQueue(int k) { data=new int[k]; cap=k; }
boolean enQueue(int v) {
if(isFull()) return false;
data[(head+size)%cap]=v; size++; return true;
}
boolean deQueue() {
if(isEmpty()) return false;
head=(head+1)%cap; size--; return true;
}
int Front() { return isEmpty()?-1:data[head]; }
int Rear() { return isEmpty()?-1:data[(head+size-1)%cap]; }
boolean isEmpty() { return size==0; }
boolean isFull() { return size==cap; }
}
```

5. Tree & Binary Search Tree

Q41. Implement all three DFS traversals (Inorder, Preorder, Postorder) iteratively.

■ **Asked at: TCS · Infosys · Wipro · All Service Co.**

■ **Pattern: Tree DFS**

Inorder: push left, visit, go right. Use explicit stack to avoid recursion.

■ **Time O(n) | Space O(h) where h = height**

```
// Inorder (Left-Root-Right) iterative
public List<Integer> inorder(TreeNode root) {
List<Integer> res = new ArrayList<>();
Deque<TreeNode> stack = new ArrayDeque<>();
TreeNode curr = root;
while (curr != null || !stack.isEmpty()) {
while (curr != null) { stack.push(curr); curr = curr.left; }
curr = stack.pop();
res.add(curr.val);
curr = curr.right;
}
return res;
}
```

Q42. Level-order traversal (BFS) of a Binary Tree.

■ **Asked at: TCS · Infosys · Cognizant · Accenture**

■ **Pattern: Tree BFS**

Use a Queue. Process level by level using size of queue at start of each iteration.

■ **Time O(n) | Space O(w) where w = max width**

```
public List<List<Integer>> levelOrder(TreeNode root) {
List<List<Integer>> res = new ArrayList<>();
if (root == null) return res;
Queue<TreeNode> q = new LinkedList<>();
q.offer(root);
while (!q.isEmpty()) {
int sz = q.size();
List<Integer> level = new ArrayList<>();
for (int i = 0; i < sz; i++) {
TreeNode node = q.poll();
level.add(node.val);
if (node.left != null) q.offer(node.left);
if (node.right != null) q.offer(node.right);
}
res.add(level);
}
return res;
}
```

Q43. Find the height (maximum depth) of a Binary Tree.

■ **Asked at: TCS · Infosys · Wipro · HCL · Capgemini**

■ **Pattern: Tree DFS / Recursion**

Recursive: max depth = 1 + max(left depth, right depth).

■ **Time O(n) | Space O(h)**

```
public int maxDepth(TreeNode root) {
if (root == null) return 0;
return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

✔ *Tip: Iterative BFS version: count levels using queue.*

Q44. Check if a Binary Tree is balanced.

■ **Asked at: TCS Digital · Wipro Elite · Cognizant**

■ **Pattern: Tree DFS / Recursion**

Bottom-up: return -1 (unbalanced) or actual height. Avoids recomputation.

■ **Time O(n) | Space O(h)**

```
public boolean isBalanced(TreeNode root) {
return checkHeight(root) != -1;
}
int checkHeight(TreeNode node) {
if (node == null) return 0;
int l = checkHeight(node.left);
int r = checkHeight(node.right);
if (l == -1 || r == -1 || Math.abs(l - r) > 1) return -1;
return 1 + Math.max(l, r);
}
```

Q45. Lowest Common Ancestor (LCA) of two nodes in a BST.

■ **Asked at: TCS Digital · Infosys SP · IBM · Cognizant**

■ **Pattern: Tree DFS**

For BST: if both keys < root go left; if both > root go right; else root is LCA.

■ **BST: O(h) | General: O(n)**

```
// BST version
public TreeNode lcaBST(TreeNode root, TreeNode p, TreeNode q) {
while (root != null) {
if (p.val < root.val && q.val < root.val) root = root.left;
else if (p.val > root.val && q.val > root.val) root = root.right;
else return root;
}
return null;
}
// General Binary Tree version
public TreeNode lca(TreeNode root, TreeNode p, TreeNode q) {
if (root==null || root==p || root==q) return root;
TreeNode l = lca(root.left,p,q), r = lca(root.right,p,q);
return (l!=null && r!=null) ? root : (l!=null ? l : r);
}
```

Q46. Validate if a Binary Tree is a valid BST.

■ **Asked at: TCS Digital · Wipro Elite · Cognizant GenC Pro**

■ **Pattern: Tree DFS**

Pass min/max bounds down the recursion. Each node must be in (min, max).

■ **Time O(n) | Space O(h)**

```
public boolean isValidBST(TreeNode root) {
return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
}
boolean validate(TreeNode n, long min, long max) {
if (n == null) return true;
if (n.val <= min || n.val >= max) return false;
return validate(n.left, min, n.val) &&
validate(n.right, n.val, max);
}
```

■ *Watch out: Common mistake: only checking left < root and root < right is NOT sufficient for subtrees.*

Q47. Right-side view of a Binary Tree.

■ **Asked at: TCS Digital · Cognizant**

■ **Pattern: Tree BFS**

BFS level by level — the last node of each level is visible from the right.

■ **Time O(n) | Space O(w)**

```
public List<Integer> rightSideView(TreeNode root) {
List<Integer> res = new ArrayList<>();
if (root == null) return res;
Queue<TreeNode> q = new LinkedList<>();
q.offer(root);
while (!q.isEmpty()) {
int sz = q.size();
for (int i = 0; i < sz; i++) {
TreeNode n = q.poll();
if (i == sz - 1) res.add(n.val);
if (n.left != null) q.offer(n.left);
if (n.right != null) q.offer(n.right);
}
}
return res;
}
```

Q48. Diameter of a Binary Tree (longest path between any two nodes).

■ **Asked at: TCS Digital · Infosys SP**

■ **Pattern: Tree DFS**

At each node: diameter through it = left height + right height. Track global max.

■ **Time O(n) | Space O(h)**

```
int maxDiam = 0;
public int diameterOfBinaryTree(TreeNode root) {
height(root); return maxDiam;
}
int height(TreeNode n) {
if (n == null) return 0;
int l = height(n.left), r = height(n.right);
maxDiam = Math.max(maxDiam, l + r);
return 1 + Math.max(l, r);
}
```

Q49. Serialize and Deserialize a Binary Tree.

■ **Asked at: TCS Digital · Infosys SP · IBM**

■ **Pattern: Tree DFS / Design**

Preorder traversal with null markers. Use split on comma for deserialization.

■ **Time O(n) | Space O(n)**

```
public String serialize(TreeNode root) {
if (root == null) return "N,";
return root.val + "," + serialize(root.left) + serialize(root.right);
}
int[] idx = {0};
public TreeNode deserialize(String data) {
String[] parts = data.split(",");
return build(parts);
}
TreeNode build(String[] p) {
if (p[idx[0]].equals("N")) { idx[0]++; return null; }
TreeNode n = new TreeNode(Integer.parseInt(p[idx[0]++]));
n.left = build(p); n.right = build(p);
return n;
}
```

Q50. Find the Kth smallest element in a BST.

■ **Asked at: TCS Digital · Wipro · Cognizant · HCL**

■ **Pattern: BST / Inorder**

Inorder traversal of BST gives sorted order. Stop at Kth element.

■ **Time O(h+k) | Space O(h)**

```
int count = 0, result = 0;
public int kthSmallest(TreeNode root, int k) {
inorder(root, k); return result;
}
void inorder(TreeNode n, int k) {
if (n == null) return;
inorder(n.left, k);
if (++count == k) { result = n.val; return; }
inorder(n.right, k);
}
```

---

6. Dynamic Programming

Q51. Fibonacci Number (top-down memoization & bottom-up tabulation).

■ **Asked at: TCS NQT · Infosys · Wipro · All Service Co.**

■ **Pattern: Dynamic Programming**

Classic DP warm-up. Bottom-up O(1) space is the optimal.

■ **Tabulation: Time O(n) Space O(1) | Memoization: O(n) O(n)**

```
// Bottom-up O(n) time O(1) space
public int fib(int n) {
if (n <= 1) return n;
int a = 0, b = 1;
for (int i = 2; i <= n; i++) { int c = a+b; a=b; b=c; }
return b;
}
// Top-down memoization
Map<Integer,Integer> memo = new HashMap<>();
public int fibMemo(int n) {
if (n<=1) return n;
return memo.computeIfAbsent(n, k -> fibMemo(k-1)+fibMemo(k-2));
}
```

Q52. Coin Change — minimum coins to make amount.

■ **Asked at: TCS Digital · Infosys SP · Cognizant GenC Pro · IBM**

■ **Pattern: Dynamic Programming — Unbounded Knapsack**

Bottom-up DP: dp[i] = min coins for amount i. Iterate all coins for each amount.

■ **Time O(amount × coins) | Space O(amount)**

```
public int coinChange(int[] coins, int amount) {
int[] dp = new int[amount + 1];
Arrays.fill(dp, amount + 1); // infinity
dp[0] = 0;
for (int i = 1; i <= amount; i++)
for (int c : coins)
if (c <= i) dp[i] = Math.min(dp[i], dp[i-c] + 1);
return dp[amount] > amount ? -1 : dp[amount];
}
// coins=[1,2,5], amount=11 -> 3 (5+5+1)
```

Q53. Longest Common Subsequence (LCS).

■ **Asked at: TCS Digital · Infosys SP · Wipro Elite**

■ **Pattern: Dynamic Programming — 2D**

2D DP table. If chars match: dp[i][j] = 1 + dp[i-1][j-1]. Else max of top/left.

■ **Time O(m×n) | Space O(m×n) -> optimize to O(n)**

```
public int lcs(String a, String b) {
int m = a.length(), n = b.length();
int[][] dp = new int[m+1][n+1];
for (int i = 1; i <= m; i++)
for (int j = 1; j <= n; j++)
dp[i][j] = (a.charAt(i-1)==b.charAt(j-1))
? 1 + dp[i-1][j-1]
: Math.max(dp[i-1][j], dp[i][j-1]);
return dp[m][n];
}
// "abcde","ace" -> 3
```

Q54. 0/1 Knapsack Problem.

■ **Asked at: TCS Digital · Cognizant · Wipro Elite · Infosys**

■ **Pattern: Dynamic Programming — Knapsack**

For each item, decide: include or exclude. dp[i][w] = max value with i items and capacity w.

■ **Time O(n×W) | Space O(n×W) -> optimize to O(W)**

```
public int knapsack(int[] wt, int[] val, int W) {
int n = wt.length;
int[][] dp = new int[n+1][W+1];
for (int i=1; i<=n; i++)
for (int w=0; w<=W; w++)
dp[i][w] = (wt[i-1]<=w)
? Math.max(dp[i-1][w], val[i-1]+dp[i-1][w-wt[i-1]])
: dp[i-1][w];
return dp[n][W];
}
```

Q55. Longest Increasing Subsequence (LIS).

■ **Asked at: TCS Digital · Infosys SP · IBM**

■ **Pattern: Dynamic Programming / Binary Search**

DP O(n^2): dp[i] = length of LIS ending at i. Binary search approach: O(n log n).

■ **Time O(n log n) | Space O(n)**

```
// O(n log n) using patience sort / binary search
public int lis(int[] nums) {
List<Integer> tails = new ArrayList<>();
for (int n : nums) {
int lo = 0, hi = tails.size();
while (lo < hi) {
int mid = (lo+hi)/2;
if (tails.get(mid) < n) lo = mid+1; else hi = mid;
}
if (lo == tails.size()) tails.add(n);
else tails.set(lo, n);
}
return tails.size();
}
// [10,9,2,5,3,7,101,18] -> 4
```

Q56. Climbing Stairs — how many ways to reach step N taking 1 or 2 steps.

■ **Asked at: TCS NQT · Infosys · Accenture · HCL · All Service Co.**

■ **Pattern: Dynamic Programming**

It's Fibonacci! dp[n] = dp[n-1] + dp[n-2].

■ **Time O(n) | Space O(1)**

```
public int climbStairs(int n) {
if (n <= 2) return n;
int a = 1, b = 2;
for (int i = 3; i <= n; i++) { int c = a+b; a=b; b=c; }
return b;
}
// n=5 -> 8 ways
```

✔ *Tip: Generalise: k steps -> dp[i] = sum of dp[i-1] to dp[i-k].*

Q57. Minimum path sum in a grid (top-left to bottom-right, only right/down moves).

■ **Asked at: TCS Digital · Wipro Elite · Cognizant**

■ **Pattern: Dynamic Programming — 2D Grid**

In-place DP: dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1]).

■ **Time O(m×n) | Space O(1) in-place**

```
public int minPathSum(int[][] grid) {
int m=grid.length, n=grid[0].length;
for (int i=0;i<m;i++)
for (int j=0;j<n;j++) {
if(i==0&&j==0) continue;
else if(i==0) grid[i][j]+=grid[i][j-1];
else if(j==0) grid[i][j]+=grid[i-1][j];
else grid[i][j]+=Math.min(grid[i-1][j],grid[i][j-1]);
}
return grid[m-1][n-1];
}
```

Q58. Word Break — can a string be segmented into dictionary words?

■ **Asked at: TCS Digital · Infosys SP · Cognizant GenC Pro**

■ **Pattern: Dynamic Programming**

dp[i] = true if s[0..i) can be segmented. Check all j < i where dp[j] && dict has s[j..i).

■ **Time O(n²) | Space O(n)**

```
public boolean wordBreak(String s, List<String> dict) {
Set<String> set = new HashSet<>(dict);
boolean[] dp = new boolean[s.length() + 1];
dp[0] = true;
for (int i = 1; i <= s.length(); i++)
for (int j = 0; j < i; j++)
if (dp[j] && set.contains(s.substring(j, i)))
{ dp[i] = true; break; }
return dp[s.length()];
}
// s="leetcode", dict=["leet","code"] -> true
```

Q59. House Robber — max sum with no two adjacent elements.

■ **Asked at: TCS Digital · Cognizant · HCL**

■ **Pattern: Dynamic Programming**

dp[i] = max(dp[i-1], dp[i-2] + nums[i]). Space optimised to two variables.

■ **Time O(n) | Space O(1)**

```
public int rob(int[] nums) {
int prev2 = 0, prev1 = 0;
for (int n : nums) {
int curr = Math.max(prev1, prev2 + n);
prev2 = prev1;
prev1 = curr;
}
return prev1;
}
// [2,7,9,3,1] -> 12 (2+9+1)
```

Q60. Edit Distance (Levenshtein Distance) between two strings.

■ **Asked at: TCS Digital · Infosys SP · IBM**

■ **Pattern: Dynamic Programming — 2D**

dp[i][j] = min ops to convert word1[0..i) to word2[0..j).

■ **Time O(m×n) | Space O(m×n)**

```
public int minDistance(String a, String b) {
int m=a.length(), n=b.length();
int[][] dp = new int[m+1][n+1];
for(int i=0;i<=m;i++) dp[i][0]=i;
for(int j=0;j<=n;j++) dp[0][j]=j;
for(int i=1;i<=m;i++)
for(int j=1;j<=n;j++)
dp[i][j] = (a.charAt(i-1)==b.charAt(j-1))
? dp[i-1][j-1]
: 1+Math.min(dp[i-1][j-1], Math.min(dp[i-1][j],dp[i][j-1]));
return dp[m][n];
}
// "horse","ros" -> 3
```

---

7. Recursion & Backtracking

Q61. Generate all subsets (Power Set) of an array.

■ **Asked at: TCS Digital · Infosys SP · Wipro Elite**

■ **Pattern: Backtracking**

At each element decide: include or exclude. Recursion builds 2^n subsets.

■ **Time O(2^n × n) | Space O(n)**

```
public List<List<Integer>> subsets(int[] nums) {
List<List<Integer>> res = new ArrayList<>();
backtrack(nums, 0, new ArrayList<>(), res);
return res;
}
void backtrack(int[] nums, int start, List<Integer> curr, List<List<Integer>> res) {
res.add(new ArrayList<>(curr));
for (int i = start; i < nums.length; i++) {
curr.add(nums[i]);
backtrack(nums, i+1, curr, res);
curr.remove(curr.size()-1); // backtrack
}
}
// [1,2,3] -> [[],[1],[1,2],[1,2,3],[1,3],[2],[2,3],[3]]
```

Q62. N-Queens problem — place N queens on N×N board with no attacks.

■ **Asked at: TCS Digital · Infosys SP**

■ **Pattern: Backtracking**

For each row place a queen in a valid column, then recurse to next row.

■ **Time O(n!) | Space O(n²)**

```
List<List<String>> results = new ArrayList<>();
public List<List<String>> solveNQueens(int n) {
char[][] board = new char[n][n];
for (char[] row : board) Arrays.fill(row, '.');
solve(board, 0, n);
return results;
}
void solve(char[][] b, int row, int n) {
if (row==n) { /* add board snapshot */ return; }
for (int col=0; col<n; col++)
if (isSafe(b,row,col,n)) {
b[row][col]='Q';
solve(b, row+1, n);
b[row][col]='.'; // backtrack
}
}
```

Q63. Combination Sum — find all combinations that sum to target (reuse allowed).

■ **Asked at: TCS Digital · Wipro Elite · Cognizant**

■ **Pattern: Backtracking**

Backtrack from each index; include same element multiple times until target hit.

■ **Time O(2^target) | Space O(target/min_candidate)**

```
public List<List<Integer>> combinationSum(int[] candidates, int target) {
List<List<Integer>> res = new ArrayList<>();
Arrays.sort(candidates);
backtrack(candidates, 0, target, new ArrayList<>(), res);
return res;
}
void backtrack(int[] c, int start, int rem, List<Integer> curr, List<List<Integer>> res) {
if (rem == 0) { res.add(new ArrayList<>(curr)); return; }
for (int i = start; i < c.length && c[i] <= rem; i++) {
curr.add(c[i]);
backtrack(c, i, rem-c[i], curr, res); // i not i+1 (reuse ok)
curr.remove(curr.size()-1);
}
}
```

Q64. Sudoku Solver.

■ **Asked at: TCS Digital · Infosys SP**

■ **Pattern: Backtracking**

For each empty cell, try digits 1-9. Check row/col/box validity. Backtrack on failure.

■ **Time O(9^m) where m = empty cells | Space O(m)**

```
public void solveSudoku(char[][] board) { solve(board); }
boolean solve(char[][] b) {
for (int r=0;r<9;r++) for (int c=0;c<9;c++)
if (b[r][c]=='.') {
for (char d='1';d<='9';d++)
if (isValid(b,r,c,d)) {
b[r][c]=d;
if(solve(b)) return true;
b[r][c]='.'; // backtrack
}
return false; // no digit worked
}
return true; // all filled
}
```

Q65. Letter Combinations of a Phone Number.

■ **Asked at: TCS Digital · Cognizant · IBM**

■ **Pattern: Backtracking**

Map digit to letters. Backtrack: append each letter and recurse on next digit.

■ **Time O(4^n × n) | Space O(n)**

```
static String[] MAP = {"","","abc","def","ghi","jkl","mno","pqrs","tuv","wxyz"};
public List<String> letterCombinations(String digits) {
List<String> res = new ArrayList<>();
if (digits.isEmpty()) return res;
backtrack(digits, 0, new StringBuilder(), res);
return res;
}
void backtrack(String digits, int i, StringBuilder sb, List<String> res) {
if (i == digits.length()) { res.add(sb.toString()); return; }
for (char c : MAP[digits.charAt(i)-'0'].toCharArray()) {
sb.append(c);
backtrack(digits, i+1, sb, res);
sb.deleteCharAt(sb.length()-1);
}
}
// "23" -> ["ad","ae",...,"cf"]
```

Q66. Flood Fill — paint connected cells of same colour.

■ **Asked at: TCS NQT · Infosys · Wipro**

■ **Pattern: DFS / Recursion**

DFS/BFS from starting cell. Change colour if it matches source colour.

■ **Time O(m×n) | Space O(m×n) recursion stack**

```
public int[][] floodFill(int[][] img, int sr, int sc, int color) {
int src = img[sr][sc];
if (src != color) dfs(img, sr, sc, src, color);
return img;
}
void dfs(int[][] img, int r, int c, int src, int color) {
if (r<0||r>=img.length||c<0||c>=img[0].length||img[r][c]!=src) return;
img[r][c] = color;
dfs(img,r+1,c,src,color); dfs(img,r-1,c,src,color);
dfs(img,r,c+1,src,color); dfs(img,r,c-1,src,color);
}
```

Q67. Number of Islands — count connected groups of '1's.

■ **Asked at: TCS Digital · Infosys · Wipro Elite · Cognizant · IBM**

■ **Pattern: DFS / Graph**

DFS from each unvisited '1', mark all connected as visited. Count starts.

■ **Time O(m×n) | Space O(m×n)**

```
public int numIslands(char[][] grid) {
int count = 0;
for (int i=0;i<grid.length;i++)
for (int j=0;j<grid[0].length;j++)
if (grid[i][j]=='1') { dfs(grid,i,j); count++; }
return count;
}
void dfs(char[][] g, int r, int c) {
if(r<0||r>=g.length||c<0||c>=g[0].length||g[r][c]!='1') return;
g[r][c]='0'; // mark visited
dfs(g,r+1,c); dfs(g,r-1,c); dfs(g,r,c+1); dfs(g,r,c-1);
}
```

✔ *Tip: Also know BFS (Queue) version — interviewers sometimes ask for both.*

8. Hashing & Collections

Q68. Group anagrams together from a list of strings.

■ **Asked at: TCS · Infosys · Cognizant · IBM · Wipro**

■ **Pattern: Hashing**

Sort each word as canonical key; group by key using HashMap.

■ **Time O(n × k log k) | Space O(n×k)**

```
public Map<String,List<String>> groupAnagrams(String[] words) {
Map<String,List<String>> map = new HashMap<>();
for (String w : words) {
char[] ch = w.toCharArray();
Arrays.sort(ch);
map.computeIfAbsent(new String(ch), k->new ArrayList<>()).add(w);
}
return map;
}
// ["eat","tea","tan","ate","nat","bat"]
// -> {aet=[eat,tea,ate], ant=[tan,nat], abt=[bat]}
```

Q69. Find the longest consecutive sequence in an unsorted array.

■ **Asked at: TCS Digital · Infosys SP · Cognizant**

■ **Pattern: Hashing**

Put all numbers in a HashSet. For each sequence start (n-1 not in set), count forward.

■ **Time O(n) | Space O(n)**

```
public int longestConsecutive(int[] nums) {
Set<Integer> set = new HashSet<>();
for (int n : nums) set.add(n);
int max = 0;
for (int n : set) {
if (!set.contains(n - 1)) { // sequence start
int len = 1;
while (set.contains(n + len)) len++;
max = Math.max(max, len);
}
}
return max;
}
// [100,4,200,1,3,2] -> 4
```

Q70. Subarray sum equals K — count subarrays with given sum.

■ **Asked at: TCS Digital · Infosys SP · IBM**

■ **Pattern: Prefix Sum + Hashing**

Prefix sum + HashMap: for each prefix[i], count how many prefix[j] = prefix[i]-k.

■ **Time O(n) | Space O(n)**

```
public int subarraySum(int[] nums, int k) {
Map<Integer,Integer> prefixCount = new HashMap<>();
prefixCount.put(0, 1);
int count = 0, sum = 0;
for (int n : nums) {
sum += n;
count += prefixCount.getOrDefault(sum - k, 0);
prefixCount.merge(sum, 1, Integer::sum);
}
return count;
}
// [1,1,1], k=2 -> 2
```

✔ *Tip: Key insight: use prefix sum difference to find subarrays in O(n).*

Q71. Top K frequent elements.

■ **Asked at: TCS Digital · Cognizant · IBM**

■ **Pattern: Heap + Hashing**

Frequency map + min-heap of size k. Or bucket sort by frequency for O(n).

■ **Time O(n log k) | Space O(n)**

```
public int[] topKFrequent(int[] nums, int k) {
Map<Integer,Integer> freq = new HashMap<>();
for (int n : nums) freq.merge(n, 1, Integer::sum);
PriorityQueue<int[]> heap = new PriorityQueue<>((a,b)->a[1]-b[1]);
for (var e : freq.entrySet()) {
heap.offer(new int[]{e.getKey(), e.getValue()});
if (heap.size() > k) heap.poll();
}
return heap.stream().mapToInt(a->a[0]).toArray();
}
// [1,1,1,2,2,3], k=2 -> [1,2]
```

Q72. Design a HashMap from scratch (without using built-in HashMap).

■ **Asked at: TCS Digital · Wipro Elite · IBM**

■ **Pattern: Hashing / Design**

Array of linked lists (chaining). Hash = key % capacity.

■ **Avg O(1) | Worst O(n) per op**

```
class MyHashMap {
static final int CAP = 1009; // prime
List<int[]>[] buckets = new List[CAP];
public void put(int key, int val) {
int idx = key % CAP;
if (buckets[idx]==null) buckets[idx]=new ArrayList<>();
for (int[] p : buckets[idx])
if(p[0]==key){p[1]=val;return;}
buckets[idx].add(new int[]{key,val});
}
public int get(int key) {
int idx=key%CAP;
if(buckets[idx]==null) return -1;
for(int[] p:buckets[idx]) if(p[0]==key) return p[1];
return -1;
}
public void remove(int key) {
int idx=key%CAP;
if(buckets[idx]!=null)
buckets[idx].removeIf(p->p[0]==key);
}
}
```

Q73. Contains Duplicate within k distance (sliding window hash).

■ **Asked at: TCS · Infosys · Wipro · HCL**

■ **Pattern: Sliding Window + Hashing**

Maintain a HashSet of size k. Slide window: add new, remove old.

■ **Time O(n) | Space O(min(n,k))**

```
public boolean containsNearbyDuplicate(int[] nums, int k) {
Set<Integer> window = new HashSet<>();
for (int i = 0; i < nums.length; i++) {
if (!window.add(nums[i])) return true; // duplicate found
if (window.size() > k) window.remove(nums[i - k]);
}
return false;
}
```

Q74. Implement Trie (Prefix Tree) — insert, search, startsWith.

■ **Asked at: TCS Digital · Infosys SP · Cognizant GenC Pro**

■ **Pattern: Trie / Design**

Each TrieNode has children[26] and a boolean isEnd flag.

■ **Insert/Search: O(L) where L=word length | Space O(ALPHABET_SIZE × L × N)**

```
class Trie {
TrieNode root = new TrieNode();
public void insert(String w) {
TrieNode n = root;
for (char c : w.toCharArray()) {
if (n.children[c-'a']==null)
n.children[c-'a']=new TrieNode();
n = n.children[c-'a'];
}
n.isEnd = true;
}
public boolean search(String w) {
TrieNode n = find(w);
return n!=null && n.isEnd;
}
public boolean startsWith(String p) { return find(p)!=null; }
private TrieNode find(String s) {
TrieNode n=root;
for(char c:s.toCharArray()){
if(n.children[c-'a']==null) return null;
n=n.children[c-'a'];
}
return n;
}
}
class TrieNode { TrieNode[] children=new TrieNode[26]; boolean isEnd; }
```

Q75. Four Sum — find all unique quadruplets summing to target.

■ **Asked at: TCS Digital · Infosys SP**

■ **Pattern: Sorting + Two Pointer**

Sort + two nested loops + two-pointer for inner pair. Skip duplicates.

■ **Time O(n³) | Space O(1)**

```
public List<List<Integer>> fourSum(int[] nums, int target) {
Arrays.sort(nums); List<List<Integer>> res = new ArrayList<>();
for (int i=0;i<nums.length-3;i++) {
if(i>0&&nums[i]==nums[i-1]) continue;
for (int j=i+1;j<nums.length-2;j++) {
if(j>i+1&&nums[j]==nums[j-1]) continue;
int l=j+1,r=nums.length-1;
while(l<r){
long s=(long)nums[i]+nums[j]+nums[l]+nums[r];
if(s==target){res.add(List.of(nums[i],nums[j],nums[l++],nums[r--]));}
else if(s<target)l++; else r--;
}
}
}
return res;
}
```

9. Java 8 Streams & Functional Programming

Q76. Find second largest number in a list using Streams.

■ **Asked at: TCS · Infosys · Wipro · Accenture · All Service Co.**

■ **Pattern: Streams / Sorting**

Distinct, sort descending, skip 1, findFirst.

■ **Time O(n log n) | Space O(n)**

```
OptionalInt secondLargest = IntStream.of(3,1,4,1,5,9,2,6)
.distinct()
.boxed()
.sorted(Comparator.reverseOrder())
.skip(1)
.mapToInt(Integer::intValue)
.findFirst();
```

Q77. Separate even and odd numbers from a list into two groups using Streams.

■ **Asked at: TCS NQT · Infosys · Wipro · Cognizant**

■ **Pattern: Streams / Collectors**

Collectors.partitioningBy returns a Map>.

■ **Time O(n) | Space O(n)**

```
Map<Boolean, List<Integer>> result = List.of(1,2,3,4,5,6)
.stream()
.collect(Collectors.partitioningBy(n -> n % 2 == 0));
// {true=[2,4,6], false=[1,3,5]}
```

Q78. Count words in a sentence using Streams (word frequency map).

■ **Asked at: TCS · Wipro · Accenture · HCL**

■ **Pattern: Streams / groupingBy**

■ **Time O(n) | Space O(k)**

```
String sentence = "the cat sat on the mat the cat";
Map<String,Long> freq = Arrays.stream(sentence.split(" "))
.collect(Collectors.groupingBy(w->w, Collectors.counting()));
// {the=3, cat=2, sat=1, on=1, mat=1}
```

Q79. Flatten a nested List> and remove duplicates using Streams.

■ **Asked at: TCS · Infosys · Wipro · Accenture**

■ **Pattern: Streams / flatMap**

■ **Time O(n log n) | Space O(n)**

```
List<List<Integer>> nested = List.of(List.of(1,2,3), List.of(2,3,4), List.of(5));
List<Integer> flat = nested.stream()
.flatMap(Collection::stream)
.distinct()
.sorted()
.collect(Collectors.toList());
// [1, 2, 3, 4, 5]
```

Q80. Sum of salaries of employees in each department using Streams.

■ **Asked at: TCS · Infosys · Cognizant · Wipro**

■ **Pattern: Streams / groupingBy + summingDouble**

groupingBy department + summingDouble for aggregation.

■ **Time O(n) | Space O(d) departments**

```
// record Employee(String dept, double salary) {}
Map<String,Double> deptSalary = employees.stream()
.collect(Collectors.groupingBy(
Employee::dept,
Collectors.summingDouble(Employee::salary)
));
```

Q81. Find employees with salary above average using Streams.

■ **Asked at: TCS Digital · Infosys · Wipro Elite**

■ **Pattern: Streams / Statistics**

Compute average first, then filter. Two-pass stream.

■ **Time O(n) | Space O(n)**

```
double avg = employees.stream()
.mapToDouble(Employee::salary).average().orElse(0);
List<Employee> aboveAvg = employees.stream()
.filter(e -> e.salary() > avg)
.collect(Collectors.toList());
```

Q82. Sort a list of strings by length, then alphabetically using Streams.

■ **Asked at: TCS NQT · Wipro · Accenture · Capgemini**

■ **Pattern: Streams / Comparator**

Chained Comparator: comparingInt(length).thenComparing(naturalOrder).

■ **Time O(n log n) | Space O(n)**

```
List<String> sorted = List.of("banana","apple","kiwi","mango","fig")
.stream()
.sorted(Comparator.comparingInt(String::length)
.thenComparing(Comparator.naturalOrder()))
.collect(Collectors.toList());
// [fig, kiwi, apple, mango, banana]
```

Q83. Convert a List to a Map (string -> its length) using Streams.

■ **Asked at: TCS · Infosys · Wipro · Accenture · HCL**

■ **Pattern: Streams / toMap**

■ **Time O(n) | Space O(n)**

```
Map<String,Integer> lengthMap = List.of("java","spring","boot")
.stream()
.collect(Collectors.toMap(
s -> s,
String::length
));
// {java=4, spring=6, boot=4}
```

■ *Watch out: Use (a,b)->a merge function if there can be duplicate keys to avoid IllegalStateException.*

Q84. Implement custom Comparator to sort Employee by salary desc, then name asc.

■ **Asked at: TCS · Infosys · Wipro · All Service Co.**

■ **Pattern: Comparator / Sorting**

■ **Time O(n log n) | Space O(1)**

```
employees.sort(
Comparator.comparingDouble(Employee::salary).reversed()
.thenComparing(Employee::name)
);
// Lambda equivalent:
employees.sort((a, b) -> {
int cmp = Double.compare(b.salary(), a.salary());
return cmp != 0 ? cmp : a.name().compareTo(b.name());
});
```

Q85. Generate an infinite stream of prime numbers and collect the first 10.

■ **Asked at: TCS Digital · Infosys SP**

■ **Pattern: Streams / Infinite Stream**

Stream.iterate starting from 2, filter by primality, limit(10).

■ **Time O(n√n) where n = 29 for first 10 primes**

```
List<Integer> primes = Stream.iterate(2, n -> n + 1)
.filter(n -> {
if (n < 2) return false;
return IntStream.rangeClosed(2, (int)Math.sqrt(n))
.noneMatch(i -> n % i == 0);
})
.limit(10)
.collect(Collectors.toList());
// [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

---

10. Concurrency & Multithreading Coding

Q86. Print numbers 1-10 alternately using two threads.

■ **Asked at: TCS Digital · Infosys SP · Wipro Elite · IBM**

■ **Pattern: Synchronization / wait-notify**

Use synchronized + wait/notify on a shared lock object.

■ **Time O(n) | Space O(1)**

```
class AlternatePrint {
static int num = 1; static final int MAX = 10;
static final Object lock = new Object();
static void print(boolean isOdd) throws InterruptedException {
while (num <= MAX) {
synchronized(lock) {
while(num<=MAX && (num%2==0)==isOdd) lock.wait();
if(num<=MAX) System.out.println(Thread.currentThread().getName()+": "+num++);
lock.notifyAll();
}
}
}
}
// Thread-1 prints odd, Thread-2 prints even numbers
```

Q87. Implement a thread-safe counter using AtomicInteger vs synchronized.

■ **Asked at: TCS · Infosys · Cognizant · HCL · Accenture**

■ **Pattern: Concurrency / Atomic Operations**

AtomicInteger uses CAS (Compare-And-Swap) at CPU level — faster than synchronized.

■ **AtomicInteger: O(1) lock-free | synchronized: O(1) with lock overhead**

```
// AtomicInteger (preferred for single counter)
class Counter {
private AtomicInteger count = new AtomicInteger(0);
public void increment() { count.incrementAndGet(); }
public int get() { return count.get(); }
}
// synchronized version
class CounterSync {
private int count = 0;
public synchronized void increment() { count++; }
public synchronized int get() { return count; }
}
```

✔ *Tip: Use LongAdder for very high contention — it keeps per-thread cells and merges on read.*

Q88. Producer-Consumer using BlockingQueue.

■ **Asked at: TCS Digital · Infosys SP · Wipro Elite · IBM**

■ **Pattern: Concurrency / BlockingQueue**

BlockingQueue.put() blocks when full; take() blocks when empty — no manual sync needed.

■ **Time O(1) per op | Space O(capacity)**

```
BlockingQueue<Integer> queue = new LinkedBlockingQueue<>(5);

Runnable producer = () -> {
for (int i=1;i<=10;i++) {
try { queue.put(i); System.out.println("Produced: "+i); }
catch (InterruptedException e) { Thread.currentThread().interrupt(); }
}
};
Runnable consumer = () -> {
for (int i=0;i<10;i++) {
try { int v=queue.take(); System.out.println("Consumed: "+v); }
catch (InterruptedException e) { Thread.currentThread().interrupt(); }
}
};
new Thread(producer).start(); new Thread(consumer).start();
```

Q89. Implement a Singleton using enum (thread-safe, serialization-safe).

■ **Asked at: TCS · Infosys · Wipro · Cognizant · All Service Co.**

■ **Pattern: Design Pattern / Concurrency**

Enum singleton is the most robust approach — JVM guarantees single instance.

■ **O(1) | Zero overhead**

```
public enum DatabaseConnection {
INSTANCE;
private final String url = "jdbc:mysql://localhost/db";
public Connection getConnection() {
// return or create connection
return null;
}
}
// Usage:
DatabaseConnection.INSTANCE.getConnection();
```

✔ *Tip: Enum singleton is immune to reflection attacks and serialization issues — Josh Bloch's recommendation.*

Q90. ForkJoin — parallel sum of a large array using RecursiveTask.

■ **Asked at: TCS Digital · Infosys SP**

■ **Pattern: Fork/Join / Divide & Conquer**

Split array in half recursively; base case sums directly; combine results.

■ **Time O(n/p) where p = processors | Space O(log n) stack**

```
class SumTask extends RecursiveTask<Long> {
int[] arr; int l, r;
static final int THRESHOLD = 1000;
SumTask(int[] a, int l, int r){arr=a;this.l=l;this.r=r;}
protected Long compute() {
if (r-l <= THRESHOLD) {
long s=0; for(int i=l;i<r;i++) s+=arr[i]; return s;
}
int mid=(l+r)/2;
SumTask left=new SumTask(arr,l,mid);
SumTask right=new SumTask(arr,mid,r);
left.fork();
return right.compute() + left.join();
}
}
// Usage: new ForkJoinPool().invoke(new SumTask(arr,0,arr.length));
```

Q91. CompletableFuture — fetch data from 3 services in parallel and combine results.

■ **Asked at: TCS Digital · Infosys SP · Wipro Elite**

■ **Pattern: Concurrency / CompletableFuture**

allOf() waits for all; thenApply combines results. Non-blocking async pipeline.

■ **Time O(max of 3 services) — true parallel | Space O(1)**

```
CompletableFuture<String> user = CompletableFuture.supplyAsync(() -> fetchUser());
CompletableFuture<String> orders = CompletableFuture.supplyAsync(() -> fetchOrders());
CompletableFuture<String> payment = CompletableFuture.supplyAsync(() -> fetchPayment());

String result = CompletableFuture.allOf(user, orders, payment)
.thenApply(v -> user.join() + "|" + orders.join() + "|" + payment.join())
.get(5, TimeUnit.SECONDS);
```

✔ *Tip: Use exceptionally() or handle() for error recovery on individual futures.*

Q92. Deadlock — demonstrate and fix using lock ordering.

■ **Asked at: TCS Digital · Infosys SP · IBM · Cognizant**

■ **Pattern: Concurrency / Deadlock Prevention**

Deadlock when Thread-A locks L1 then L2, Thread-B locks L2 then L1 simultaneously.

■ **Prevention: no performance cost if designed upfront**

```
// DEADLOCK (bad)
// Thread-A: synchronized(lock1) { synchronized(lock2) { ... } }
// Thread-B: synchronized(lock2) { synchronized(lock1) { ... } }

// FIX: Always acquire locks in the same order
void transfer(Account from, Account to, int amt) {
Account first = from.id < to.id ? from : to; // consistent order
Account second = from.id < to.id ? to : from;
synchronized(first) {
synchronized(second) {
from.balance -= amt;
to.balance += amt;
}
}
}
```

✔ *Tip: Alternative: use tryLock() with timeout from ReentrantLock to detect and retry.*

11. Design Patterns (Coding Round)

Q93. Implement the Observer Pattern (Event/Listener system).

■ **Asked at: TCS Digital · Infosys SP · IBM**

■ **Pattern: Observer Pattern**

Subject maintains a list of Observers; notifies all on state change.

■ **Subscribe O(1) | Publish O(n observers)**

```
interface Observer { void update(String event); }

class EventBus {
private Map<String, List<Observer>> listeners = new HashMap<>();
public void subscribe(String type, Observer o)
{ listeners.computeIfAbsent(type, k->new ArrayList<>()).add(o); }
public void publish(String type, String data)
{ listeners.getOrDefault(type, List.of()).forEach(o->o.update(data)); }
}

// Usage
EventBus bus = new EventBus();
bus.subscribe("ORDER_PLACED", e -> System.out.println("Email: "+e));
bus.subscribe("ORDER_PLACED", e -> System.out.println("SMS: "+e));
bus.publish("ORDER_PLACED", "Order #123");
```

Q94. Implement the Builder Pattern for an immutable Order object.

■ **Asked at: TCS · Infosys · Wipro · All Service Co.**

■ **Pattern: Builder Pattern**

Nested static Builder class with fluent setters; build() creates immutable object.

■ **O(1) build | O(n) fields**

```
public final class Order {
private final String id; private final String product;
private final int qty; private final double price;
private Order(Builder b){id=b.id;product=b.product;qty=b.qty;price=b.price;}
public static class Builder {
private String id,product; private int qty; private double price;
public Builder id(String v){id=v;return this;}
public Builder product(String v){product=v;return this;}
public Builder qty(int v){qty=v;return this;}
public Builder price(double v){price=v;return this;}
public Order build(){return new Order(this);}
}
}
// Usage:
Order o = new Order.Builder().id("1").product("Book").qty(2).price(19.99).build();
```

Q95. Implement Strategy Pattern for payment processing (UPI, Card, Wallet).

■ **Asked at: TCS Digital · Infosys SP · Cognizant**

■ **Pattern: Strategy Pattern**

Strategy interface + concrete implementations + context class.

■ **O(1) | Easily extensible (Open/Closed Principle)**

```
interface PaymentStrategy {
void pay(double amount);
}
class UPIPayment implements PaymentStrategy {
public void pay(double amt) { System.out.println("Paid ■"+amt+" via UPI"); }
}
class CardPayment implements PaymentStrategy {
public void pay(double amt) { System.out.println("Paid ■"+amt+" via Card"); }
}
class PaymentContext {
private PaymentStrategy strategy;
public void setStrategy(PaymentStrategy s) { strategy = s; }
public void checkout(double amt) { strategy.pay(amt); }
}
// ctx.setStrategy(new UPIPayment()); ctx.checkout(499.0);
```

Q96. Implement the Decorator Pattern to add logging & caching to a service.

■ **Asked at: TCS Digital · Wipro Elite**

■ **Pattern: Decorator Pattern**

Wrap the original service with decorator classes that add behaviour.

■ **O(1) per decoration layer**

```
interface DataService { String fetchData(String key); }
class DatabaseService implements DataService {
public String fetchData(String key) { return "DB_"+key; }
}
class CachingDecorator implements DataService {
private final DataService wrapped;
private final Map<String,String> cache = new HashMap<>();
CachingDecorator(DataService s){wrapped=s;}
public String fetchData(String k)
{ return cache.computeIfAbsent(k, wrapped::fetchData); }
}
class LoggingDecorator implements DataService {
private final DataService wrapped;
LoggingDecorator(DataService s){wrapped=s;}
public String fetchData(String k){
System.out.println("Fetching: "+k);
return wrapped.fetchData(k);
}
}
// DataService svc = new LoggingDecorator(new CachingDecorator(new DatabaseService()));
```

Q97. Implement Factory Method Pattern for shape creation.

■ **Asked at: TCS · Infosys · Wipro · Cognizant · HCL**

■ **Pattern: Factory Pattern**

Factory method returns appropriate subclass based on type string.

■ **O(1)**

```
interface Shape { double area(); }
record Circle(double r) implements Shape { public double area(){return Math.PI*r*r;} }
record Rectangle(double w,double h) implements Shape { public double area(){return w*h;} }

class ShapeFactory {
public static Shape create(String type, double... dims) {
return switch(type.toLowerCase()) {
case "circle" -> new Circle(dims[0]);
case "rectangle" -> new Rectangle(dims[0], dims[1]);
default -> throw new IllegalArgumentException("Unknown: "+type);
};
}
}
// ShapeFactory.create("circle", 5.0).area() -> 78.54
```

12. SQL & Database Coding

Q98. Find the second highest salary from an Employee table.

■ **Asked at: TCS NQT · Infosys · Wipro · All Service Co.**

■ **Pattern: SQL / Window Functions**

Use LIMIT OFFSET or subquery with MAX.

■ **O(n log n)**

```
-- Approach 1: LIMIT OFFSET
SELECT salary FROM Employee
ORDER BY salary DESC
LIMIT 1 OFFSET 1;

-- Approach 2: Subquery (handles NULLs, ANSI SQL)
SELECT MAX(salary) FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);

-- Approach 3: DENSE_RANK (handles ties correctly)
SELECT salary FROM (
SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
FROM Employee
) t WHERE rnk = 2;
```

✔ *Tip: DENSE_RANK is the most correct solution when there are duplicate salaries.*

Q99. Find employees who earn more than their manager.

■ **Asked at: TCS Digital · Infosys SP · Cognizant · IBM**

■ **Pattern: SQL / Self Join**

Self-join on managerId = id.

■ **O(n) with index on managerId**

```
SELECT e.name AS Employee
FROM Employee e
JOIN Employee m ON e.managerId = m.id
WHERE e.salary > m.salary;
```

Q100. Find departments with no employees.

■ **Asked at: TCS · Infosys · Wipro · Accenture**

■ **Pattern: SQL / Outer Join**

LEFT JOIN + NULL check, or NOT IN / NOT EXISTS.

■ **O(n + m) with index**

```
-- LEFT JOIN approach
SELECT d.name FROM Department d
LEFT JOIN Employee e ON d.id = e.deptId
WHERE e.id IS NULL;

-- NOT EXISTS approach (better for large tables)
SELECT name FROM Department d
WHERE NOT EXISTS (
SELECT 1 FROM Employee WHERE deptId = d.id
);
```

✔ *Tip: Prefer NOT EXISTS over NOT IN when the subquery column can be NULL.*

Q101. Find the Nth highest salary using DENSE_RANK.

■ **Asked at: TCS Digital · Infosys SP · Wipro Elite**

■ **Pattern: SQL / Window Functions**

Window function DENSE_RANK handles ties; wrap in subquery to filter rank = N.

■ **O(n log n)**

```
SELECT salary
FROM (
SELECT salary,
DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
FROM Employee
) ranked
WHERE rnk = :N; -- replace :N with desired rank
```

Q102. Find duplicate rows in a table.

■ **Asked at: TCS · Infosys · Wipro · Cognizant · All Service Co.**

■ **Pattern: SQL / GROUP BY / HAVING**

GROUP BY the columns you want to check, HAVING COUNT > 1.

■ **O(n log n)**

```
-- Find duplicate emails
SELECT email, COUNT(*) AS cnt
FROM Person
GROUP BY email
HAVING COUNT(*) > 1;

-- Delete duplicates, keep lowest id
DELETE FROM Person
WHERE id NOT IN (
SELECT MIN(id) FROM Person GROUP BY email
);
```

✔ *Tip: Always clarify with interviewer: keep min id or max id when deleting duplicates.*
